

# AI-Native Readiness Report — ai-integration

Level 1 — Assisted

|            |                                                                                    |
|------------|------------------------------------------------------------------------------------|
| Repository | drleewarden/ai-integration                                                         |
| URL        | local path ( /Users/darrynlee-warden/Documents/DEV/ai-integration )                |
| Branch     | feat/cta-boxes (1 styling commit ahead of main; shared checkout, audited in place) |
| Commit     | b0ddd23                                                                            |
| Audited    | 2026-07-26                                                                         |
| Audited by | ai-native-readiness v1                                                             |

## Executive summary

**Level 1 — Assisted.** The fundamentals an agent needs day-to-day are genuinely strong — strict TypeScript, a fast 253-test suite that runs green, clean PR-based history, and rich agent context files (CLAUDE.md, AGENTS.md). What holds the repo back is that nothing *enforces* any of it: there is no CI at all, so lint and tests are never run on a proposed change, and the two files a fresh clone needs for determinism — a lockfile and the env template — are both deliberately gitignored. A single small CI workflow plus committing `package-lock.json` and an env template would lift this repo to Level 2.

A check scored **Partial** still counts toward the level — Partials are the improvement backlog, not blockers. Only a **Fail** holds a level back.

## Scorecard

| Check                           | Level | Score   |
|---------------------------------|-------|---------|
| R1 — Version control hygiene    | 1     | Pass    |
| R2 — One-command build and test | 1     | Partial |

|                                 |   |         |
|---------------------------------|---|---------|
| R3 — Navigable structure        | 1 | Pass    |
| R4 — Docs an agent can use      | 1 | Partial |
| R5 — Reviewable change flow     | 1 | Partial |
| R6 — Linting and formatting     | 2 | Partial |
| R7 — Type safety                | 2 | Pass    |
| R8 — Meaningful test suite      | 2 | Pass    |
| R9 — Continuous integration     | 2 | Fail    |
| R10 — Reproducible environment  | 2 | Partial |
| R11 — Dependency currency       | 2 | Partial |
| R12 — Automated deployment path | 3 | Pass    |
| R13 — Safe failure and rollback | 3 | Partial |
| R14 — Production observability  | 3 | Fail    |

## The level ladder

| Level | Name          | Requires                             | Status                              |
|-------|---------------|--------------------------------------|-------------------------------------|
| 1     | Assisted      | R1–R5 all $\geq$ Partial             | ✓ reached                           |
| 2     | Collaborative | Level 1 + R6–R11 all $\geq$ Partial  | ✗ blocked by R9 (no CI)             |
| 3     | Autonomous    | Level 2 + R12–R14 all $\geq$ Partial | ✗ blocked by R14 (no observability) |

## Findings

### R1 — Version control hygiene • Pass

**Evidence:** Incremental, intent-bearing conventional-commit history ( `feat(members)` : `member_activity` table with RLS ); every recent change lands via PR (#4–#20); secret scan of tracked content clean, no `.env` files tracked, `.gitignore` covers `env/build/deps/` `.pem` . Branch protection is off (GitHub API returns 404), but a consistent PR-merge flow substitutes per this check's bar.

## R2 — One-command build and test • Partial

**Evidence:** `npm run build` → exit 0 (full Next.js production build, 40+ routes) and `npm test` → exit 0 (30 suites, 253 tests, 2.2s), exactly as documented in `package.json/README/CLAUDE.md`.

**Gap:** `package-lock.json` is explicitly gitignored, so no machine — including an agent sandbox or CI — can reproduce the dependency tree the build was verified against. Commit a lockfile.

## R3 — Navigable structure • Pass

**Evidence:** Textbook App Router layout: route groups `(main)` / `(dark-nav)` / `(members)` own shared chrome, API routes are one-directory-one-purpose, domain logic lives in `lib/` (the readiness scoring pipeline is fully isolated), content in `content/insights/*.json`. The two largest files (`CalculatorClient.tsx` at 1,487 lines, `workshop-melbourne` at 1,182) are copy-heavy page JSX, not tangled core logic.

## R4 — Docs an agent can use • Partial

**Evidence:** README covers setup/run/test accurately for named scripts; CLAUDE.md (93 lines) and AGENTS.md (127 lines) encode architecture, conventions, env vars and security gotchas — exactly the context files this check wants.

**Gap:** README directs newcomers to `.env.example`, but that file is untracked (swallowed by the `.env*` gitignore pattern) — a fresh clone hits a dead reference immediately. CLAUDE.md also describes the test suite as "thin" (3 suites) when 30 suites exist. Track the env template under a name the ignore pattern misses (e.g. `env.example`) and refresh the testing section.

## R5 — Reviewable change flow • Partial

**Evidence:** All recent work merges via PRs; many are small and focused (+7/-2, +11/-11).

**Gap:** Several PRs are unreviewably large (#17: +10,246/-9; #11: +4,357) and there's no PR template or CODEOWNERS (no `.github/` directory exists). For agent contributions, large self-merged PRs are where mistakes slip through — cap PR size and add a template with a checklist.

## R6 — Linting and formatting • Partial

**Evidence:** ESLint 9 flat config with next/core-web-vitals + TypeScript rules; tracked code is near-clean (only 4 files flagged; 24 of 28 flagged files are untracked `.claude/worktrees/` leftovers).

**Gap:** The documented `npm run lint` exits 1 as-run because local worktree copies aren't ignored, no formatter is configured, and nothing enforces lint anywhere (no CI, no pre-commit hooks). Add `.claude/` to ESLint ignores and put lint in the CI gate.

## R7 — Type safety • Pass

**Evidence:** `strict: true` TypeScript across ~27k lines with near-zero escape hatches (1 suppression comment, 3 loose `any` s in app/lib); type-checking runs on every `next build`, which passed clean.

## R8 — Meaningful test suite • Pass

**Evidence:** 30 suites / 253 tests / 381 assertions, green in 2.2 seconds. Coverage lands where it matters: the readiness scoring pipeline (the business-critical algorithm), members access rules, and the API surface (send-email, stripe-webhook, members-download, security-headers). Real behavioural assertions throughout.

## R9 — Continuous integration • Fail

**Evidence:** There is no `.github/` directory and no CI config of any kind. All 14+ merged PRs landed with no automated build/lint/test verdict — Vercel's deploy build is the only machine check, and it runs *after* merge.

**Gap:** This is the single blocker for Level 2. One GitHub Actions workflow running `npm ci && npm run lint && npx jest --ci && npm run build` on every PR closes it.

## R10 — Reproducible environment • Partial

**Evidence:** Runtime pinned twice ( `.nvmrc` , `engines: 22.x` ); env vars are thoroughly documented in a well-commented `.env.example` and listed in CLAUDE.md/AGENTS.md.

**Gap:** The `.env.example` file itself is untracked and there is no committed lockfile — a fresh clone (which is all an agent sandbox ever is) gets the right Node but a floating dependency tree and no env template file.

## R11 — Dependency currency • Partial

**Evidence:** As of 2026-07-26 every major is current or in support: Next.js 15.5.15, React 19.2.5, Node 22.x, ESLint 9, Jest 30.

**Gap:** No Renovate/Dependabot, no bump-commit habit, and Tailwind is pinned to a `4.0.0-alpha.29` prerelease. Versions are fresh because the project is young, not because a process keeps them fresh. Without a lockfile, `npm audit` can't even run.

## R12 — Automated deployment path • Pass

**Evidence:** GitHub deployments API shows vercel[bot] creating Production deployments on every push to main and Preview deployments per branch (most recent: 2026-07-23). Merge → production is fully automated with no manual step; no manual deploy procedure is documented anywhere.

## R13 — Safe failure and rollback • Partial

**Evidence:** Vercel offers platform-native instant rollback to any previous immutable deployment.

**Gap:** That path is undocumented and unexercised in this repo, and Supabase migrations are forward-only with known state drift (bootstrap SQLs exist because migration 0001 fails on fresh projects) — a bad migration can only be fixed forward. Document the rollback runbook and adopt expand-contract migration discipline.

## R14 — Production observability • Fail

**Evidence:** No monitoring or error-tracking SDK in the dependency tree (no Sentry/Datadog/OTel/Rollbar), no alerting config, no uptime checks, no health endpoint. Ten API route files log via `console.error` only, visible solely to whoever browses Vercel function logs. GTM/GA4 is product analytics, not observability.

**Gap:** A production regression is currently noticed by a customer, not a system. Sentry's free tier plus one uptime check on the readiness-submit endpoint would flip this.

## Recommendations

1. **Add a CI workflow gating every PR** ( `.github/workflows/ci.yml` : `npm ci` , `lint` , `jest --ci` , `next build` ) — CI is the referee between an agent's "done, tests pass" and reality, and it is the only thing blocking Level 2. *Effort: Small.*
2. **Commit `package-lock.json`** (remove it from `.gitignore` , run `npm install` , commit) — without it neither CI nor an agent sandbox can reproduce the tree your green build ran against. *Effort: Small.*
3. **Track the env template** (rename to `env.example` or add `!.env.example` to `.gitignore` ) and fix the stale CLAUDE.md testing note — a fresh agent clone currently hits a dead README reference in its first five minutes. *Effort: Small.*
4. **Silence the false-positive lint noise and enforce lint in CI** (ignore `.claude/` in `eslint.config.mjs` ) — the documented lint command must exit 0 for an agent to trust it as a check. *Effort: Small.*
5. **Add error tracking + one uptime check** (e.g. Sentry SDK in the API routes, uptime monitor on `/api/readiness/submit` ) — unlocks progress toward Level 3; today a

regression waits for a human to notice. *Effort: Medium.*

6. **Set up Renovate or Dependabot** — keeps majors supported without relying on the project staying young; also restores `npm audit` signal once a lockfile exists. *Effort: Small.*
7. **Document the rollback runbook and tighten migration discipline** (Vercel instant-rollback steps; expand-contract for Supabase changes; reconcile the migration/bootstrap drift) — bounded blast radius is the price of autonomous shipping. *Effort: Medium.*
8. **Add a PR template and keep PRs reviewable** — +10k-line PRs can't absorb agent contributions safely; the flow exists, the discipline needs a guardrail. *Effort: Small.*

## Effort rollup

| Priority | Recommendation                          | Effort |
|----------|-----------------------------------------|--------|
| 1        | CI workflow on every PR                 | S      |
| 2        | Commit the lockfile                     | S      |
| 3        | Track env template, refresh stale docs  | S      |
| 4        | Clean + enforce lint                    | S      |
| 5        | Error tracking + uptime check           | M      |
| 6        | Renovate/Dependabot                     | S      |
| 7        | Rollback runbook + migration discipline | M      |
| 8        | PR template + size discipline           | S      |

**Overall:** Four small fixes — CI, lockfile, env template, lint hygiene — take this repo from Level 1 to Level 2; one medium observability effort opens the road to Level 3.